



Chapter 7 — Simulation Engine Design

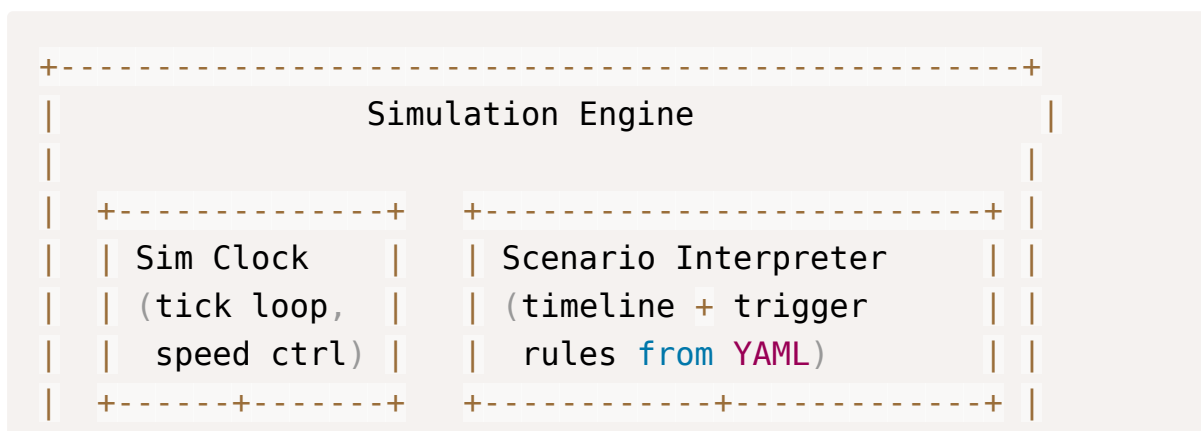


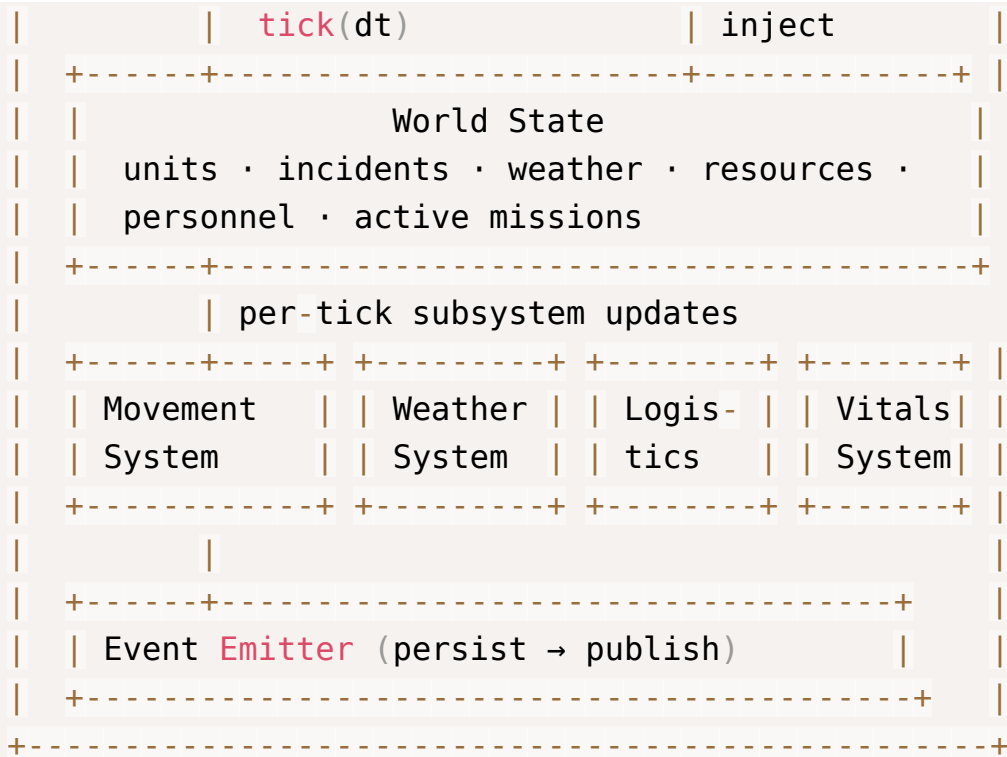
The simulation engine is **the real core product** of DOIP. Every other module — map, dashboards, AI, logistics, health, weather, analytics — is a consumer of the events this engine produces. It must be designed first and designed well.

7.1 Responsibilities

- Maintain the authoritative world state of a run: units, positions, incidents, weather, resources, personnel vitals.
- Advance the world on a fixed **tick loop** with adjustable speed (1x, 5x, 20x real time).
- Execute **scenario scripts** (scheduled and conditional events).
- Generate **synthetic sensor streams** (GPS, environmental readings, unit reports) with realistic noise.
- Emit every state change as a typed event to the DB and event bus.
- Support **pause/resume/stop**, deterministic seeding, and replay.

7.2 Engine Architecture





Tick loop (asyncio):

```

async def run_loop(self):
    TICK = 1.0 # 1 sim-second per tick at speed 1.0
    while self.state.status == "running":
        dt = TICK * self.state.speed
        self.clock.advance(dt)
        self.scenario.evaluate(self.clock.now) # scripted
+ conditional injects
        self.movement.update(dt) # units f
ollow routes/behaviors
        self.weather.update(dt) # cells d
rift, intensity evolves
        self.logistics.update(dt) # consump
tion from activity
        self.vitals.update(dt) # synthe
tic vitals per person
        await self.emitter.flush() # batch p
ersist + publish
        await self.sleep_to_wall_clock() # keeps r
eal-time pacing

```

7.3 Scenario Definition Format (YAML)

Scenarios are the authoring surface for training exercises — human-readable, diffable, versionable.

```
name: "Flood Response – Sector Exercise 1"
area: "geo/ops_area.geojson"
sim_start: "2026-07-21T06:00:00Z"
duration_minutes: 180
seed: 42

entities:
  - id: team_alpha
    type: ground_team
    callsign: "Team Alpha"
    start: [77.591, 12.972]
    behavior: patrol # patrol | idle | follow_ro
    ute | respond
    patrol_zone: sector_a
    personnel: 6
  - id: drone_1
    type: drone
    callsign: "Drone Alpha"
    start: [77.60, 12.98]
    battery_pct: 100
    behavior: idle
  - id: depot_north
    type: depot
    start: [77.58, 12.99]
    resources: { fuel: 1200, medkit: 40, food: 300, batter
y: 20 }

timeline: # scheduled events (sim-ti
me offsets)
  - at: "+00:10"
    action: spawn_weather
    params: { kind: rain, intensity: 0.7, center: [77.62, 1
2.96],
            radius_km: 3, drift_kmh: [5, -2] }
```

```

- at: "+00:25"
  action: create_incident
  params: { kind: medical, severity: critical,
            title: "Evacuation requested", near: team_bra
vo }
- at: "+00:40"
  action: unit_report
  params: { unit: drone_1, text: "Vehicle movement observ
ed near bridge" }

rules:                                     # conditional triggers (ev
aluated each tick)
- when: "unit_in_weather(team_bravo, rain, 0.5)"
  then: { action: modify_speed, params: { unit: team_brav
o, multiplier: 0.6 } }
- when: "resource_below(depot_north, medkit, 0.3)"
  then: { action: raise_alert, params: { severity: warnin
g,
            title: "Medkit stock below 30% at North Depot"
} }
- when: "random_chance(0.002)"           # per tick, seeded RNG
  then: { action: create_incident, params: { kind: obstru
ction, severity: info } }

```

Design notes

- `timeline` = deterministic script; `rules` = reactive dynamics; both feed the same action executor.
- All randomness flows through one seeded RNG → same seed = identical run (testable, replayable, gradeable).
- Condition DSL is a small whitelisted function set — no arbitrary code execution in scenario files.

7.4 Movement System

Behavior	Logic
<code>follow_route</code>	Interpolate along mission LineString at unit speed × terrain × weather multipliers; emit <code>WaypointReached</code>

Behavior	Logic
<code>patrol</code>	Random waypoints within assigned zone (seeded), dwell pauses
<code>respond</code>	On tasking, compute route to incident via mission service, then <code>follow_route</code>
<code>idle</code>	Small GPS jitter (realistic noise), battery drain if drone airborne

Speed model: `effective_speed = base_speed × terrain_factor × weather_factor × health_factor`, every factor documented and surfaced in the UI (transparency requirement FR-9.3).

7.5 Synthetic Sensor & Report Generation

- **GPS:** true position + Gaussian noise ($\sigma \approx 5$ m); occasional dropout windows to simulate degraded comms.
- **Environmental:** temperature/wind/visibility sampled from the weather field at the unit's location + noise.
- **Unit reports:** template library ("{unit} observed {observation} near {landmark}") filled from world state — giving the AI service *realistic*, *imperfect* text to summarize.
- **Vitals:** per-person baseline + circadian drift + activity response (moving raises HR) + scripted anomaly injections.

7.6 Determinism, Persistence & Replay

1. Every tick's changes are flushed as one batch: `events` insert + projection updates in a single transaction.
2. Position history down-sampled to 1 sample/5 sim-seconds into `unit_positions`.
3. **Replay mode** re-reads events between two sim-times and streams them to the client at chosen speed — the engine itself is not needed for replay.
4. **Crash recovery:** on restart, engine reloads world state from projections and resumes from `sim_clock`.

7.7 Performance Targets

Metric	Target
Entities per run	200

Metric	Target
Tick rate	1 Hz sim-tick (positions interpolated client-side for smoothness)
Event throughput	50 events/sec sustained burst
Tick budget	< 50 ms per tick at full load (leaves headroom for 20x speed)

7.8 Testing the Engine

- **Golden-run tests:** run scenario seed 42 for 100 ticks → assert exact event sequence hash.
- **Property tests:** units never leave the operational area; battery never negative; resources conserve mass.
- **Speed invariance:** 20x speed produces the same event sequence as 1x (only wall-clock differs).
- **Rule engine tests:** each DSL function unit-tested; malformed scenario YAML rejected with line-level errors.